Modul 5

Datensicherheit

Josef Altmann



Der vorliegende Foliensatz basiert auf:
Kemper, A., Leis, V., Eickler, A.: *Datenbanksysteme*, De Gruyter Oldenbourg, 11. Auflage (2026)
Saake, G., Sattler, K.-U., Heuer, A.: *Datenbanken: Konzepte und Sprachen*, MITP, 6. Auflage (2018)

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

© 2026

Sicherheitsaspekte
… im Datenbankkontext

- Bisher …
  - Schutz von Daten vor **unabsichtlicher Beschädigung** durch
    - Integritätsbedingungen
      - » Entitätsintegrität (Primärschlüssel)
      - » referentielle Integrität (Fremdschlüssel)
      - » semantische Integrität (Constraints, …)
    - Transaktionsverwaltung (ACID-Prinzipien, …)
    - Mehrbenutzersynchronisation (Sperr-basierte Verfahren, …)
- Im folgenden …
  - Schutz gegen **absichtliche Beschädigung** und **unbefugte Nutzung und Manipulation** durch
    - Identifikation und Authentisierung (Benutzername, Passwort, …)
    - Autorisierung und Zugriffskontrolle (Zugriffsrechte vergeben/überprüfen)
    - Zugriffskontrolle in SQL (Datenzugriff einschränken)
    - Auditing (Überwachung und Protokollierung von Aktionen)
    - SQL-Injection (Einschleusen von schadhaften SQL-Anweisungen)

© 2026

Datenbanksicherheit

- **Systemsicherheit**
  - Zugriff und Verwendung der Datenbank auf Systemebene
    - Benutzername und Passwort
    - Plattenspeicher, der den Benutzern zugewiesen wird
    - Systemoperationen, welche die Benutzer ausführen können
- **Datensicherheit**
  - Zugriff auf und die Verwendung der Datenbankobjekte sowie die Aktionen, die Benutzer für die Objekte ausführen können.

- Die Benutzer benötigen
  - **Systemrechte**, um auf die Datenbank zugreifen zu können
  - **Objektrechte**, um den Inhalt von Datenbankobjekten zu bearbeiten.

© 2026

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

© 2026

Sicherheitsmodelle

- Frage:
  - Wie kann man erreichen, dass Benutzer mit unterschiedlichen (System-/Objekt-)Rechten gemeinsam auf Daten zugreifen können?
- Antwort:
  - Einführung von Sicherheitsmodelle (⇨ **Zugriffskontrolle**)
    - Discretionary Access Control – DAC (= benutzerdefiniert)
      - » Eigentümer vergibt Zugriffsrechte auf Objekte
      - » Standard bei Datenbank- und Betriebssystemen
    - Mandatory Access Control – MAC (= vorgeschrieben)
      - » Zugriffsrechte zentral vom System (Subjekte und Objekte werden Sicherheitsklassen zugeordnet - Policy)
      - » Eigentümer kann Zugriffsrechte nicht verändern
    - Role Based Access Control – RBAC (= rollenbasiert)
      - » vereinfachte Administration
      - » auch in Kombination mit DAC und MAC
      - » Benutzer erhalten Zugriffsrechte über Rollenmitgliedschaft(en)

© 2026

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

© 2026

Zugriffskontrolle in SQL

Discretionary Access Control 1/2

- Explizite Autorisierung durch Regeln:
  - Der Zugriff auf ein **Objekt** o kann nur erfolgen, wenn für das **Subjekt** s ein **Zugriffsrecht** r vorliegt
  - Autorisierungsregel: $(o, s, r) \rightarrow \{\text{ja}, \text{nein}\}$

- Schutzinformation
  - **Subjekt:** Initiator der Handlung
    - Benutzer, Gruppen, Programme, Rechner, Geräte, ...
  - **Objekt:** Gegenstand der Handlung
    - Relationen, Tupel, Attribute, Sichten, ...
    - Programme, Prozesse, ...
  - **Zugriffsrecht:** Was darf gemacht werden
    - lesen, ändern, ausführen, erzeugen, …
    - weitergeben/entziehen von Zugriffsrechten, ...

Zugriffskontrolle in SQL

Discretionary Access Control 2/2

- Schutzzustand eines DBS zum Zeitpunkt `t` wird durch Matrix `M(t)` modelliert:

Subjekte, Benutzer	Objekte				
	o_1	o_2	o_3	...	o_n
s_1	r_1, r_2	–	r_3	–	r_3, r_4
s_2	–	r_2, r_3	–	–	r_4
...	...	...	...	...	...
s_n	r_2	–	r_5	–	r_i, r_k

- Zugriffsmatrix ist sehr groß und dünn besetzt
- DAC ist einfach, weit verbreitet und in SQL realisiert
- Eigentümer (meist Ersteller) ist immer für Rechtevergabe verantwortlich

Zugriffskontrolle in SQL

- Autorisierung
  - Zentrale Vergabe der Zugriffsrechte (Datenbankadministrator)
  - Dezentrale Vergabe der Zugriffsrechte (Eigentümer)
- Objektgranulat
  - Wertunabhängige oder
  - Wertabhängige Objektfestlegung (Sichtkonzept)
- Wirksamkeit der Zugriffskontrolle beruht auf drei Annahmen:
  - Fehlerfreie Benutzer-Identifikation/-Authentisierung
  - Erfolgreiche Abwehr von Eindringlingen
    - strikte Isolation der Benutzer- und DBS-Prozesse
    - Schutz der Datenübertragung
  - Schutzinformation (= Zugriffsmatrix) ist hochgradig geschützt

© 2026

Zugriffskontrolle in SQL
Systemrechte

- Der Datenbankadministrator (DBA) besitzt Systemrechte auf höchster Ebene für Aufgaben wie:

  - **CREATE** [ANY] TABLE/VIEW/TYPE/INDEX/CLUSTER/TRIGGER/PROCEDURE
    Benutzer darf die entsprechenden Schema-Objekte erzeugen

  - **ALTER** [ANY] TABLE/TYPE/TRIGGER/PROCEDURE
    Benutzer darf die entsprechenden Schemaobjekte verändern

  - **DROP** [ANY] TABLE/TYPE/VIEW/INDEX/CLUSTER/TRIGGER/PROCEDURE
    Benutzer darf die entsprechenden Schema-Objekte löschen

  - **SELECT/INSERT/UPDATE/DELETE** [ANY] TABLE
    Benutzer darf in Tabellen Tupel lesen/erzeugen/verändern/entfernen

ANY berechtigt dazu die entsprechende Operation in jedem Schema auszuführen, fehlt **ANY**, so bleibt die Ausführung auf das eigene Schema beschränkt.

© 2026

Zugriffskontrolle in SQL

Systemrechte – Identifikation und Authentisierung

- Der DBA erstellt Benutzer mit der `CREATE USER`-Anweisung

Notation

```
CREATE USER benutzer
IDENTIFIED BY passwort;
```

- Später z.B. Passwort ändern

Notation

```
ALTER USER benutzer
IDENTIFIED BY "new_passwort";
```

Zugriffskontrolle in SQL

Systemrechte – Vergabe

- Nachdem ein Benutzer erstellt wurde, kann der DBA dem Benutzer bestimmte Systemrechte erteilen.

Notation

```
GRANT <Systemrecht> (, <Systemrecht>)*
TO (<Benutzer> (, <Benutzer>)*) | PUBLIC;
```

- `systemrechte_liste`:
  - **CREATE SESSION**
  - **CREATE TABLE**
  - **CREATE VIEW**
  - **CREATE PROCEDURE**
  - **CREATE SEQUENCE**
  - ...

Beispiel

```
GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW
TO altmann;
```

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

Zugriffskontrolle in SQL

Objektrechte – Autorisierung und Zugriffskontrolle

- Objektrecht ist eine Berechtigung, eine bestimmte Aktion für eine bestimmte Tabelle, Sicht etc. auszuführen.
  - Objektrechte variieren von Objekt zu Objekt
  - Eigentümer (= Erzeuger) hat alle Rechte für ein Objekt

Objektrechte	Tabelle	Sicht	Prozedur	Sequence
ALTER	✓			✓
DELETE	✓	✓		
EXECUTE			✓	
INDEX	✓			
INSERT	✓	✓		
SELECT	✓	✓		✓
UPDATE	✓	✓		
REFERENCES	✓			

© 2026

Zugriffskontrolle in SQL

Objektrechte – Wirkungsweise

- Besitzer hat die **volle Verfügungsgewalt** über die eigenen ("privaten") Datenbankobjekte (Lesen, Ändern, …).
- Andere Benutzer haben per Default **keine Zugriffsrechte** auf Datenbankobjekte anderer Benutzer.
- Ein Benutzer kann Zugriffsrechte auf seine "privaten" Datenbankobjekte an andere Benutzer **weitergeben**.

- Die Art des Zugriffsrechts kann **beschränkt** werden (z.B. auf lesenden Zugriff).
- Beschränkung des Lesezugriffs auf ausgewählte Attribute mittels **`VIEW`-Mechanismus**.
- Beschränkung des Änderungsrechts (`UPDATE`) auf ausgewählte **Attribute** durch Angabe der Attribute in der `UPDATE`-Klausel.

- Erteilung der **Erlaubnis zur Weitergabe** von Zugriffsrechten
  ⇨ `WITH GRANT OPTION`.

Zugriffskontrolle in SQL

Objektrechte – SQL-Syntax

$$\left\{\begin{array}{l}\underline{\textbf{GRANT}}\\ \underline{\textbf{REVOKE}}\end{array}\right\} \left\{\begin{array}{l}\underline{\textbf{ALL PRIVILEGES}}\\ <privilege> \left[\text{, } <privilege>\text{, } \ldots\right]\end{array}\right\} \underline{\textbf{ON}} \left\{\begin{array}{l}[\ \underline{\textbf{TABLE}}\ ]\ \textit{relview}\\ \underline{\textbf{DOMAIN}}\ \textit{domain}\\ \ldots\end{array}\right\}$$

$$\left\{\begin{array}{l}\underline{\text{TO}}\\ \underline{\text{FROM}}\end{array}\right\} \left\{\begin{array}{l}\underline{\textbf{PUBLIC}}\\ \textit{userid1}\ [, \textit{userid2}, \ldots]\end{array}\right\} [\ \underline{\textbf{WITH GRANT OPTION}}\ ]^{*)}$$

$$<privilege> ::= \left\{\begin{array}{l}\underline{\textbf{SELECT}}\\ \underline{\textbf{DELETE}}\\ \underline{\textbf{INSERT}}\ [(\textit{Attrib1}\ [, \textit{Attrib2}, \ldots])]\\ \underline{\textbf{UPDATE}}\ [(\textit{Attrib1}\ [, \textit{Attrib2}, \ldots])]\\ \underline{\textbf{REFERENCES}}\ [(\textit{Attrib1}\ [, \textit{Attrib2}, \ldots])]\\ \ldots\end{array}\right\}$$

*) nur in Verbindung mit GRANT wählbar

Zugriffskontrolle in SQL

Objektrechte – Vergabe

Notation

```
GRANT ( <Objektrechte> | ALL PRIVILEGES )
   ON <Objekt>
   TO ( <Benutzer/Rollen> | PUBLIC )
[ WITH GRANT OPTION ];
```

- Erläuterungen:
  - Objektrechte: `ALL PRIVILEGES` oder selektiv aus Liste: **`SELECT`, `INSERT`, `UPDATE`, `DELETE`** etc.
  - `ON`: Schemaobjekt, für das die Privilegien vergeben werden (z.B. Tabelle, Sicht)
  - `TO`: Autorisierungsidentifikator (auch `PUBLIC` möglich)
  - `WITH GRANT OPTION`: Recht auf Weitergabe von Rechten; Eigentümer des Schemaobjekts besitzen implizit dieses Recht.

Zugriffskontrolle in SQL

Objektrechte – Vergabe

Beispiel

Erteilen Sie den Lesezugriff auf die Relation `Professoren` an die Benutzer `Altmann` und `Pitzer`.

```
GRANT SELECT
   ON Professor
   TO Altmann, Pitzer;
```

Beispiel

Erteilen Sie das Recht zur Aktualisierung der Attribute `MatrNr`, `VorlNr`, `PersNr` in der Relation `pruefen` an den Benutzer `Altmann`.

```
GRANT UPDATE (MatrNr, VorlNr, PersNr)
   ON pruefen
   TO Altmann;
```

© 2026

Zugriffskontrolle in SQL

Objektrechte – Sichten

- Mit `GRANT`-Befehl können Objektrechte für Sichten vergeben werden
  - `INSERT`, `UPDATE` und `DELETE`-Rechte machen nur Sinn, wenn die Sicht aktualisierbar ist!

- **Benutzer** der Sicht braucht **keine Rechte an den Basistabellen**, die der Sicht zugrunde liegen
  - Benutzer kann nur über die Sicht zugreifen
  - nur einen Teil der Zeilen oder Spalten bzw.
  - nur aggregierte oder anonymisierte Daten sehen

- Ersteller der Sicht braucht Rechte auf Basistabelle zum Abfragezeitpunkt

© 2026

Zugriffskontrolle in SQL

Objektrechte – Autorisierung einer Sicht

- Rechte von Bedingungen abhängig machen durch den Einsatz von Sichten

Beispiel

TutorInnen sollen für die Studierenden des ersten Semesters die Daten lesen dürfen, aber nicht die Daten der anderen Studierenden.

```
CREATE VIEW BetreuteStudenten AS
  SELECT *
    FROM Student
   WHERE Semester = 1;

GRANT SELECT
   ON BetreuteStudenten
   TO Brayer, Herzog;
```

© 2026

Zugriffskontrolle in SQL

Objektrechte – Autorisierung einer individuellen Sicht

Beispiel

Die Studierenden sollen ihre zugeordneten Prüfung lesen dürfen.

```
CREATE VIEW StudentenNotenSicht AS
SELECT p.*
  FROM pruefen p
 WHERE EXISTS (SELECT s.*
                 FROM Studenten s
                WHERE s.MatrNr = p.MatrNr
                  AND s.user = USER);

GRANT SELECT
   ON StudentenNotenSicht
   TO PUBLIC;
```

Vordefinierte Pseudospalte

Zugriffskontrolle in SQL

Objektrechte – Entzug

Notation

```
REVOKE ( <Objektrechte> | ALL PRIVILEGES )
    ON <Objekt>
  FROM ( <Benutzer/Rollen> | PUBLIC )
[ RESTRICT | CASCADE ];
```

- Autorisierungsgraph notwendig, wenn Objektrecht mit `GRANT OPTION` weitergegeben wurde
  - `RESTRICT`: Falls Objektrecht bereits an Dritte weitergegeben ⇨ Abbruch von **`REVOKE`**
  - `CASCADE`: Rücknahme des Objektrechts mittels **`REVOKE`** an alle Benutzer propagiert, die es von diesem Benutzer mit **`GRANT`** erhalten haben.

Hinweis: `RESTRICT` und `CASCADE` sind so in Oracle nicht verfügbar!

```
REVOKE ( <Objektrechte> | ALL PRIVILEGES ) ON <Objekt>
FROM ( <Benutzer/Rollen> | PUBLIC ) CASCADE CONSTRAINTS;
```

Zugriffskontrolle in SQL

Objektrechte – Entzug

Beispiel

Rücknahme des Lesezugriffes der Relation `Professoren` von den Benutzern `Altmann` und `Pitzer`.

```
REVOKE SELECT
    ON Professor
  FROM Altmann, Pitzer CASCADE;
```

Überblick

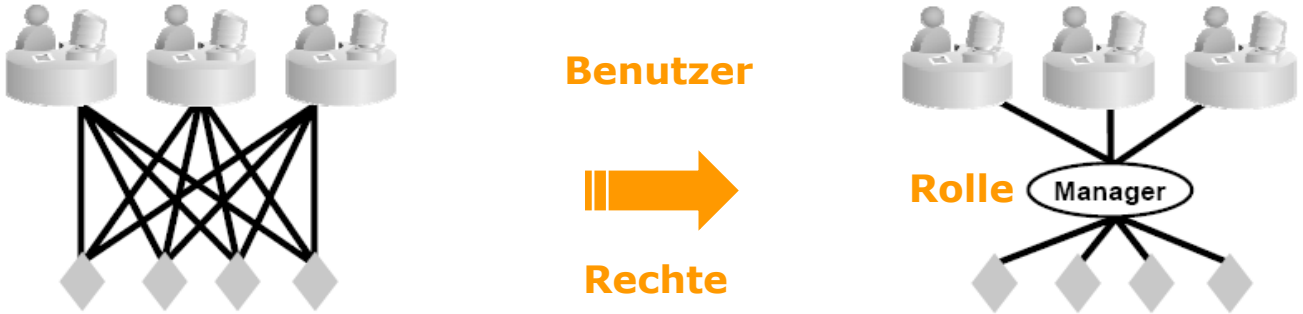
- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

© 2026

Zugriffskontrolle in SQL

Role Based Access Control – Rollenbasierte Autorisierung

- Bei der rollenbasierten Zugriffskontrolle werden die Rechte nicht mehr direkt an Benutzer, sondern an Rollen geknüpft.



  - Rollen entsprechen häufig Funktionen/Aufgaben, die in einer Organisation (Bank, Behörde, Unternehmen) auftreten.
  - Geeignet für hierarchisch aufgebaute Organisationen.

- Rollenkonzept vereinfacht Berechtigungsmanagement

Zugriffskontrolle in SQL

Rollenbasierte Autorisierung – Arbeiten mit Rollen

Rolle erstellen

```
CREATE ROLE Student;
CREATE ROLE Professor;
```

Rolle Systemrechte zuweisen

```
GRANT CREATE TABLE, CREATE VIEW TO Student;
```

Rolle Objektrechte zuweisen

```
GRANT SELECT ON Vorlesung TO Student;
```

Rolle Benutzer zuweisen

```
GRANT Student TO S1010456001;
[ WITH ADMIN OPTION ]   ⟶ Erlaubt die Weitergabe und das Löschen
```

Rolle einer Rolle zuweisen

```
GRANT Student TO Professor;
```

Rolle entziehen

```
REVOKE Student FROM S1010456001;
```

© 2026

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
  - Discretionary Access Control
    - Systemrechte: Zugriff auf die Datenbank
    - Objektrechte: Inhalt der Datenbankobjekte bearbeiten
  - Rollenbasierte Autorisierung
- Auditing
- SQL-Injection

Zugriffskontrolle in SQL
Auditing

- Operationen von Benutzern auf Datenbankobjekten werden aufgezeichnet
- Auditfolge (= aufgezeichnete Daten) kann helfen, Sicherheitslücken aufzudecken
  - Bspw. fehlgeschlagene Zugriffsversuche

Beispiel

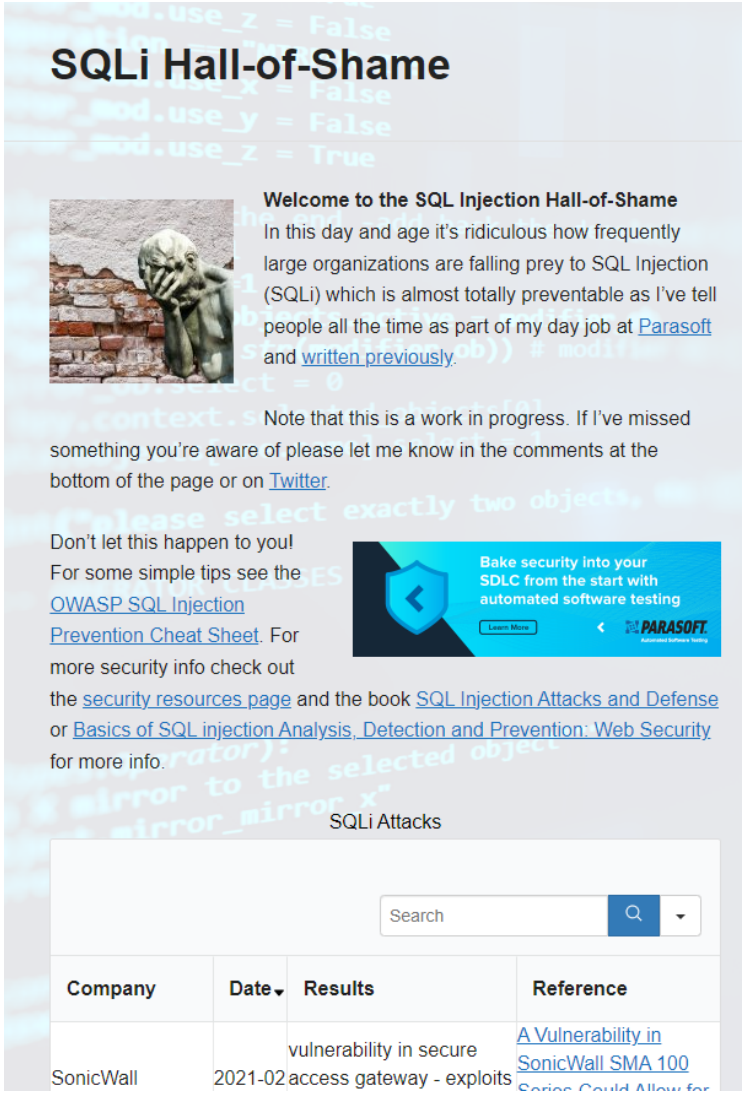
```
AUDIT INSERT, UPDATE, DELETE
   ON Professor
  WHENEVER [NOT] SUCCESSFUL;
NOAUDIT INSERT, UPDATE, DELETE ON Professor;
```

- Prinzip "Mehr Überwachung = Mehr Sicherheit" mit Vorsicht anwenden
  - Zusatzaufwand für Protokollierung verlangsamt Operationen
  - Große Protokolldateien schwer analysierbar

© 2026

Überblick

- Sicherheitsaspekte
- Sicherheitsmodelle
- Zugriffskontrolle in SQL
- Auditing
- SQL-Injection



siehe: https://codecurmudgeon.com/wp/sql-injection-hall-of-shame/

SQL-Injection

- Einschleusen (Injizieren) von schadhaften SQL-Anweisungen
  - aus Benutzereingabe
  - ausführbare SQL-Anweisungen
  - ohne ausreichende Prüfung bzw. Bereinigung (Maskierung)
- Eingabeparameter können somit zu potentiell schadhaften SQL-Anweisungen führen



siehe: https://de.wikipedia.org/wiki/SQL-Injection

SQL-Injection

- SQL besitzt Metazeichen mit besonderer Bedeutung
  - **Apostroph**, um Textdaten zu umschließen
  - **Anführungszeichen**, um Textdaten zu umschließen (nur einige Datenbanken)
  - **umgekehrte Schrägstrich** (Backslash), um das nachfolgende Zeichen nicht als besonderes Zeichen zu interpretieren (Escape-Mechanismus)
  - **Semikolon**, um einen Datenbankbefehl von dem nächsten Datenbankbefehl zu trennen
  - **doppelter Bindestrich**, um den Rest der Zeile zu ignorieren (Kommentar)
- SQL-Injection nutzt diese Metazeichen zum Einschleusen von schadhaften SQL-Anweisungen.

© 2026

SQL-Injection

- SQL-Injection nutzt diese Metazeichen zum Einschleusen von schadhaften SQL-Anweisungen, um
  - **logische Verknüpfungen** zu ändern,
  - **Teilabfragen** zu entfernen,
  - **zusätzliche Abfragen** zu erzeugen,
  - den **Datenbankprozess** zu beeinflussen und
  - verräterische **Fehlermeldungen** zu erzeugen.

© 2026

SQL-Injection

Beispiel: Naive Authentifizierung

Student

MatrNr	Name	Semester	Passwort
24002	Xenokrates	18	AlterGrieche
25403	Jonas	12	Bruno
26120	Fichte	10	Idealismus
26830	Aristoxenos	8	Halbton
27550	Schopenhauer	6	WilleUndVorstellung
27551	Schopenhauer	2	Bruder
28106	Carnap	3	logischeAnalyse
29120	Theophrastos	2	Peripatos
29555	Feuerbach	2	Naturalismus

Pruefen

MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	2
27551	5041	2125	5
27550	4630	2137	2

© 2026

SQL-Injection
Naive Authentifizierung

Reguläre Anfrage übermittelt das Passwort:

```
SELECT *
  FROM Student s
       INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
 WHERE s.Name = 'Schopenhauer',
   AND s.Passwort= 'WilleUndVorstellung';
```



MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2

Hinweis: Passwort sollte natürlich nicht in Klartext übermittelt werden → verschlüsselt mittels SSL!

SQL-Injection
Naive Authentifizierung

Reguläre Anfrage übermittelt das Passwort:

```
SELECT *
  FROM Student s
       INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
 WHERE s.Name = 'Schopenhauer',
   AND s.Passwort = 'WilleUndVorstellung';
```

Anfrage im Quelltext (z.B. C):

```
sprintf(&query, "SELECT * \
  FROM Student s \
       INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr) \
 WHERE s.Name = '%s' \
   AND s.Passwort = '%s'", name, password);
```

Variabler Anteil der SQL-Anweisung

SQL-Injection
Attacke

Benutzereingabe:

Name: Schopenhauer

Passwort: WilleUndVorstellung' OR '1' = '1

Keine Validierung der Nutzereingaben!

Anfrage übermittelt Passwort und manipulierte/eingeschleuste Anweisung:

```
SELECT *
  FROM Student s
       INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
 WHERE s.Name = 'Schopenhauer'
   AND s.Passwort= 'WilleUndVorstellung' OR '1' = '1';
```



MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
25403	Jonas	12	Bruno	25403	5041	2125	2
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2
27551	Schopenhauer	2	Bruder	27551	5041	2125	5
28106	Carnap	3	logischeAnalyse	28106	5001	2126	1

© 2026

Web-Anbindung von Datenbanken

am Beispiel von Java, Servlets und JDBC

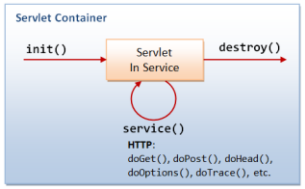


Web Client
(meist Browser)

Internet

Web Server

Application Server



Servlet-Engine

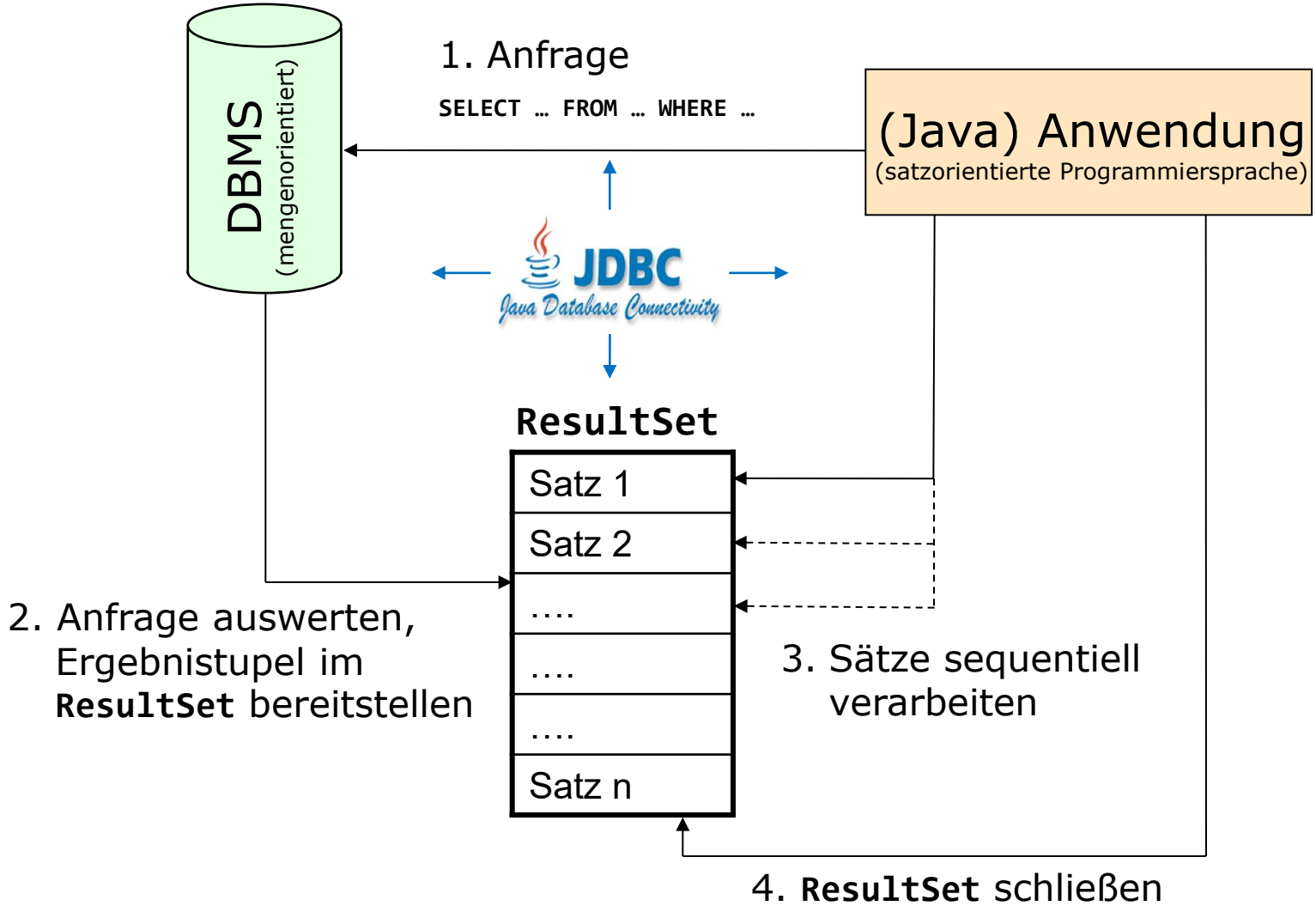
JDBC
Java Database Connectivity

SQL-Injection

Datenbank

Web-Anbindung von Datenbanken

am Beispiel von Java und JDBC



Web-Anbindung von Datenbanken

am Beispiel von Java und JDBC

```
//ACHTUNG: Als Vereinfachung nur rudimentäre Fehlerbehandlung!
public static void main(final String[] args) throws SQLException, ClassNotFoundException
{ //Datenbanktreiber laden
  Class.forName("oracle.jdbc.driver.OracleDriver");
  //Datenbankverbindungseinstellungen definieren
  final String dbUrl="jdbc:oracle:thin:@serverName:port:databaseName";
  final String dbUsername = "User";
  final String dbPassword = "Password";
  //SQL-Anweisung definieren
  final String sqlQuery = "SELECT * FROM Student";
  Connection connection = null;
  Statement statement = null;
  ResultSet resultSet = null;
  try {    //Datenbankverbindung aufbauen
           connection =   DriverManager.getConnection(dbUrl, dbUsername, dbPassword);
           //SQL-Befehlsobjekt erstellen
           statement = connection.createStatement();
           //SQL-Abfrage ausführen
           resultSet = statement.executeQuery(sqlQuery);
           //Ergebnismenge auswerten und speichern
           final Set<String> names = new TreeSet<>();
           while (resultSet.next()) {
             names.add(resultSet.getString("Nachname")); }
           //Weiterverarbeitung der ermittelten Daten
           names.forEach(name -> System.out.println("Nachname= " + name);
  }
  finally { //Achtung: Als Vereinfachung nur rudimentäre Aufräumarbeiten
    if (resultSet!=null) resultSet.close();
    if (statement!=null) statement.close();
    if (connection!=null) connection.close();}
}
```

Schwachstelle, wenn SQL-Anweisung variable Anteile aufweist!

SQL-Injection
via Web-Schnittstelle

Name:	**Schopenhauer**
Passwort:	**WilleUndVorstellung**

Reguläre Anfrage mit **variablen Teilen** in der SQL-Anweisung:

```
...
//Auslesen aus der Session/Login-Dialog etc. = Benutzereingabe
final String dbUsername = "Schopenhauer";
final String dbPassword = "WilleUndVorstellung";
...
//SQL-Anweisung definieren
final String sqlQuery =
     "SELECT * " +
     "FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr) " +
     "WHERE s.Name = '" + dbUsername + "' " +
     "AND s.Passwort = '" + dbPassword + "'";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

SQL-Syntax: einzelne Hochkommata als Begrenzer von Strings

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = 'Schopenhauer'
AND s.Passwort= 'WilleUndVorstellung';
```

MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2

© 2026

SQL-Injection
via Web-Schnittstelle

Name:	Schopenhauer
Passwort:	weissIchNichtAberEgal' **OR '1' = '1**

Hochkomma beendet String und schleust SQL-Anweisung ein:

```
...
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "Schopenhauer";
final String dbPassword = "weissIchNichtAberEgal' OR '1' = '1";
...
//SQL-Anweisung definieren
final String sqlQuery =
     "SELECT * " +
     "FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr) " +
     "WHERE s.Name = '" + dbUsername + "' " +
     "AND s.Passwort = '" + dbPassword + "'";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = 'Schopenhauer'
AND s.Passwort= 'weissIchNichtAberEgal' OR '1' = '1';
```

Schadhafte/eingeschleuste SQL-Anweisung

MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
25403	Jonas	12	Bruno	25403	5041	2125	2
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2
27551	Schopenhauer	2	Bruder	27551	5041	2125	5
28106	Carnap	3	logischeAnalyse	28106	5001	2126	1

SQL-Injection
via Web-Schnittstelle

Name:	Schopenhauer'--
Passwort:	WilleUndVorstellung

Kommentar deaktiviert restliche SQL-Anweisung:

```
...
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "Schopenhauer'--";
final String dbPassword = "WilleUndVorstellung";
...
//SQL-Anweisung definieren
final String sqlQuery =
     "SELECT * " +
     "FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr) " +
     "WHERE s.Name = '" + dbUsername + "' " +
     "AND s.Passwort = '" + dbPassword + "'";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = 'Schopenhauer'--'AND s.Passwort='WilleUndVorstellung';
```

MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2
27551	Schopenhauer	2	Bruder	27551	5041	2125	5

© 2026

SQL-Injection
via Web-Schnittstelle

WHERE-Bedingung umgehen:

```
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "' OR 1=1 --";
final String dbPassword = "...";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = '' OR 1=1 -- AND ...;
```

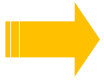


MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
25403	Jonas	12	Bruno	25403	5041	2125	2
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2
27551	Schopenhauer	2	Bruder	27551	5041	2125	5
28106	Carnap	3	logischeAnalyse	28106	5001	2126	1

Abfragebedingung auf andere Spalte umlenken:

```
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "' OR Semester > 1 --";
final String dbPassword = "...";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = '' OR Semester > 1 -- AND ...;
```



MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note
25403	Jonas	12	Bruno	25403	5041	2125	2
27550	Schopenhauer	6	WilleUndVorstellung	27550	4630	2137	2
27551	Schopenhauer	2	Bruder	27551	5041	2125	5
28106	Carnap	3	logischeAnalyse	28106	5001	2126	1

© 2026

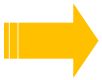
SQL-Injection
via Web-Schnittstelle

Name:	Schopenhauer
Passwort:	**NN'; DELETE FROM Pruefen WHERE 1=1 --**

Löschanweisung in der **WHERE**-Klausel durch Semikolon einschleusen:

```
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "Schopenhauer";
final String dbPassword = "NN'; DELETE FROM Pruefen WHERE 1=1 --";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = 'Schopenhauer'
AND s.Passwort= 'NN'; DELETE FROM Pruefen WHERE 1=1 --';
```



MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note

Pruefen

MatrNr	VorlNr	PersNr	Note

SQL-Injection
via Web-Schnittstelle

Name: Schopenhauer

Passwort: NN'; UPDATE Pruefen SET Note = 1.0 WHERE MatrNr=25403 --

Manipulationsanweisung in der **WHERE**-Klausel durch Semikolon einschleusen:

```
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "Schopenhauer";
final String dbPassword = "NN'; UPDATE Pruefen SET Note = 1.0
                                                WHERE MatrNr = 25403 --";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT *
FROM Student s INNER JOIN Pruefen p ON (s.MatrNr = p.MatrNr)
WHERE s.Name = 'Schopenhauer'
AND s.Passwort= 'NN';
UPDATE Pruefen SET Note = 1.0 WHERE MatrNr = 25403 --';
```

MatrNr	Name	Semester	Passwort	MatrNr	VorlNr	PersNr	Note

Pruefen

MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	**1**
27551	5041	2125	5
27550	4630	2137	2

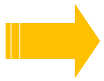
SQL-Injection
via Web-Schnittstelle

Name:	Schopenhauer
Passwort:	**NN' UNION SELECT MatrNr, Name, Semester FROM Pruefen --**

Anfrageanweisung in der **WHERE**-Klausel durch Semikolon einschleusen:

```
//Auslesen aus der Session etc. = Benutzereingabe
final String dbUsername = "Schopenhauer";
final String dbPassword = "NN' UNION SELECT MatrNr, Name, Semester FROM Student --";
...
//SQL-Anweisung definieren
final String sqlQuery =
      "SELECT MatrNr, Name, Semester " +
      "FROM Student " +
      "WHERE Name = '" + dbUsername + "' " +
      "AND Passwort = '" + dbPassword + "'";
...
//SQL-Abfrage ausführen
resultSet = statement.executeQuery(sqlQuery);
...
```

```
SELECT MatrNr, Name, Semester
FROM Student
WHERE Name = 'Schopenhauer'
AND Passwort= 'NN'
UNION
SELECT MatrNr, Name, Semester FROM Student --';
```



Student

MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
27551	Schopenhauer	2
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Ausspähen von Daten mit UNION

Schutz vor SQL-Injection-Attacken

All Input is Evil! Jede Eingabe ist böse!

Goldene Regel: **Traue niemals den Benutzereingaben!**

- Überprüfe/**Validiere Benutzereingaben**
- Filtere und **maskiere Metadaten**/-zeichen (`; ' " \`)
- Verwende konsequent parametrisierte Anfragen (gebundene Parameter in **Prepared Statements**)
- Verwende **Stored Procedures**
- Verwende **SQL-Firewalls**
- Schränke Autorisierungskorridore ein
  - Benutzer sollte nur jene System- und Objektrechte besitzen, die er/sie tatsächlich benötigt

© 2026

Schutz vor SQL-Injection-Attacken

Einsatz von parametrisierten Anfragen

Prepared Statement

```
//Benutzereingaben
final String vorlesungsTitel = "Logik";
final int persNr = 2125;

//SQL-Anweisung definieren
final String sqlQuery = "SELECT *
                        FROM Vorlesung v INNER JOIN Professor p
                             ON (v.gelesenVon = p.PersNr)
                        WHERE v.Titel = ? AND p.PersNr = ? ";

//SQL-PreparedStatement erstellen
final PreparedStatement preparedStatement =
            connection.preparedStatement(sqlQuery);
//Parameterübergabe
preparedStatement.setString(1, vorlesungsTitel);
preparedStatement.setInt(2, persNr);

//SQL-Anfrage durchführen
resultSet = preparedStatement.executeQuery();
```

Bind-Parameter

Prototyp der SQL-Anweisung

Ausführungsplan wird erstellt

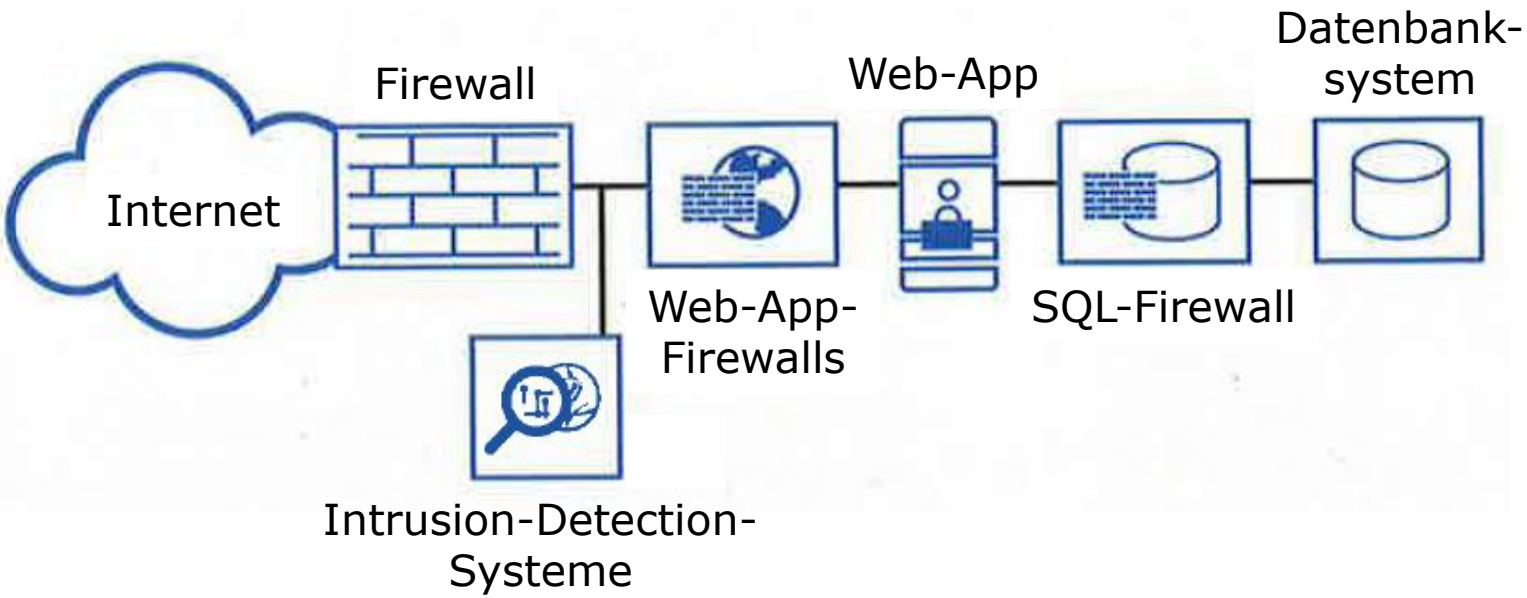
Typsichere Methoden

Schutz vor SQL-Injection-Attacken

Einsatz von SQL-Firewalls

Freie SQL-Firewalls:

- DBShield
- AcraCensor
- ProxySQL



SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP
Open Web Application Security Project

- Internationale Non-Profit-Organisation, die sich der Sicherheit von Webanwendungen widmet
  - https://owasp.org/www-project-top-ten/



- Risikokategorie „Injection“ umfasst
  - NoSQL-, OS-Command-, LDAP- und SQL-Einschleusungen
  - https://owasp.org/Top10/A03_2021-Injection/

Review-Fragen

- Sicherheitsmodelle?
- Arten von Rechten?
  - Beispiele?
- Weitergabe von Rechten?
- Auditing? Immer alles Protokollieren?
- SQL-Injection?

© 2026